

# 1 RE:SOURCE — Intelligent Resource Utilization & Decision System

Turn resources into intelligence.

RE:SOURCE is a domain-independent decision-support prototype. It accepts an organization's own resource and demand records, calculates utilization and remaining capacity, detects hidden capacity, ranks possible matches, explains every score, and simulates what happens when a resource is unavailable.

The product is intentionally **upload-first**. It does not ship with manufacturing, college, hospital, or any other organization's preloaded records. A fresh workspace begins empty. The user must choose a **Resource File** and a separate **Demand File** from the PC file manager. Until both files are accepted, every result view remains at zero or shows a source-file gate. Manual customization remains available for adding individual records.

## 1.1 1. What problem does RE:SOURCE solve?

Organizations usually have resources and requirements, but their information is fragmented across spreadsheets, ERP systems, MIS applications, emails, registers, and IoT sources. A team may request a new machine, room, vehicle, employee group, bed, laboratory, or software licence without knowing that usable capacity already exists elsewhere in the same organization.

The conventional workflow is data collection, spreadsheet cleaning, manual calculation, meetings, and a decision. RE:SOURCE creates an intelligence layer over that workflow. It changes the starting question from “What should we buy?” to “Can existing capacity satisfy this demand first?”

## 1.2 2. What makes the idea different?

RE:SOURCE is not positioned as a generic AI dashboard. It combines four ideas in one workflow:

| Idea                              | Meaning in the product                                                                                                 |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Resource reuse before acquisition | Search existing unused capacity before recommending procurement or expansion.                                          |
| Domain independence               | The data model is based on capacity, utilization, demand, context, and priority rather than one industry's vocabulary. |
| Explainable decisions             | Every match includes type fit, available capacity, location/context fit, and priority reasoning.                       |
| Digital what-if simulation        | Management can remove a resource virtually and see capacity lost, demands affected, alternatives, and risk.            |

AWS and similar cloud platforms primarily provide infrastructure and managed services. RE:SOURCE is an application-level intelligence workflow that can run over data from any organization and can itself be hosted on AWS, Azure, Google Cloud, an institutional server, or another platform. Therefore, AWS

is not the competitor at the same product layer. The distinction is: **cloud platform versus domain-independent resource decision intelligence**.

### 1.3 3. Current prototype behavior

The current prototype starts with an empty workspace. The login screen protects the demo workspace. The demo credentials are judge and resource123. Incorrect credentials display an explicit error. Successful login opens the workspace, and logout clears the demo session and returns to login.

The **Add your data** screen provides three paths. The first path is a Resource File picker. The second path is a Demand File picker. The third path is manual customization for one resource or one demand. Each picker opens the local PC file manager. The uploader accepts any file type at the browser boundary. JSON, CSV, TSV, and plain-text tables are parsed automatically when their columns can be recognized. PDF, DOCX, XLSX, images, and other binary files are accepted as source files, but the prototype does not silently pretend to understand their internal content; production ingestion should add a format extractor or a human mapping step for those files.

The analysis gate is deliberate. A resource upload alone is not sufficient, and a demand upload alone is not sufficient. The Overview, Resource inventory, Demand intelligence, Daily analysis, What-if simulator, and Daily report remain empty or locked until both source flags are ready. After both uploads succeed, the backend returns the uploaded records and all result views use those records as their only source.

### 1.4 4. Data model

A resource represents available organizational capacity. A demand represents a requirement that may be satisfied by one or more resources.

| Resource field | Purpose                                                                   |
|----------------|---------------------------------------------------------------------------|
| name           | Human-readable resource name.                                             |
| type           | Machine, workforce, facility, vehicle, software, or domain-specific type. |
| location       | Department, campus, unit, ward, farm, hub, or other context.              |
| capacity       | Total usable capacity in the selected period.                             |
| unit           | Hours, seats, beds, trips, units, people, licences, and so on.            |
| utilization    | Percentage of capacity currently used.                                    |
| owner          | Team, department, or responsible group.                                   |

| Demand field | Purpose                                                                   |
|--------------|---------------------------------------------------------------------------|
| title        | Requirement name or order description.                                    |
| requester    | Department or person requesting the capacity.                             |
| category     | Production, machining, education, quality, logistics, or custom category. |

| Demand field | Purpose                     |
|--------------|-----------------------------|
| quantity     | Required amount.            |
| unit         | Unit of the demand.         |
| priority     | High, Medium, or Low.       |
| due          | Deadline or time window.    |
| location     | Required operating context. |

## 1.5 5. Calculation theory

The prototype calculates used capacity as:

```
Used capacity = Total capacity × Utilization percentage ÷ 100
Available capacity = Total capacity – Used capacity
```

Average utilization is calculated as total used capacity divided by total capacity. Hidden capacity is the sum of available capacity from resources below the underutilization threshold. The threshold is currently 60% for the prototype visualization and can become a workspace setting in production.

The matching score is intentionally explainable. The prototype combines four signals:

```
Match score = Type compatibility × 0.45
              + Capacity fit × 0.30
              + Location/context fit × 0.15
              + Priority adjustment
```

The exact weights are prototype configuration, not a universal law. A production implementation should learn or optimize weights using historical outcomes, expert review, and governance. The system shows the factors rather than hiding them behind a black box.

## 1.6 6. Why the prototype calls itself AI-assisted

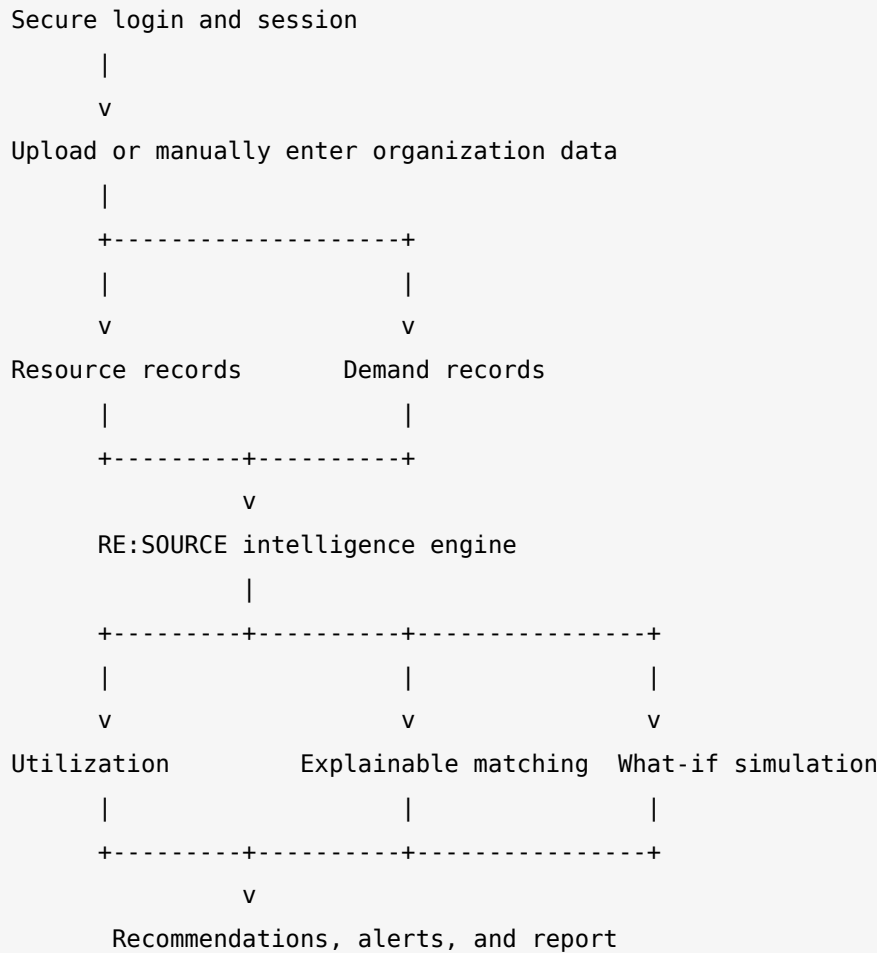
The current prototype is a working proof of concept, not a claim that a trained model has already learned every organization. It demonstrates an intelligence pipeline using deterministic, auditable rules. This is appropriate for a first SIH round because a judge can verify the calculation and understand why an answer was produced.

The production intelligence roadmap can add demand forecasting, time-series analysis, anomaly detection, optimization, natural-language extraction from unstructured documents, similarity matching, and IoT telemetry. A model should be added only where it improves a measurable decision. A human decision-maker remains in the loop for high-impact actions.

## 1.7 7. System architecture

```
Organization user
```

```
|
v
```



The frontend is a React application. The backend is an Express server with tRPC procedures. tRPC provides typed contracts between the frontend and backend. The current prototype keeps the editable demo workspace in process memory so the SIH flow is fast and easy to demonstrate. Production should store organizations, users, files, resources, demands, analysis runs, and audit events in a database.

## 1.8 8. Technology stack and why each technology is used

| Technology                  | Role          | Why it is used                                                                           |
|-----------------------------|---------------|------------------------------------------------------------------------------------------|
| React                       | Frontend UI   | Component-based interface for dashboard screens and forms.                               |
| TypeScript                  | Language      | Catches incorrect data shapes before runtime and shares types between client and server. |
| Tailwind CSS and custom CSS | Visual system | Responsive layout, consistent tokens, spacing, typography, and dashboard styling.        |

| Technology                      | Role                        | Why it is used                                                                      |
|---------------------------------|-----------------------------|-------------------------------------------------------------------------------------|
| Express                         | Backend runtime             | Hosts the application server and API integration layer.                             |
| tRPC                            | API contract                | Provides typed queries and mutations without duplicating REST contracts.            |
| Zod                             | Input validation            | Validates uploaded and manually entered fields before analysis.                     |
| Vitest                          | Automated tests             | Verifies authentication, blank-start behavior, import behavior, and scenario logic. |
| Browser FileReader API          | Upload parsing              | Reads user-selected files in the browser for the prototype import workflow.         |
| Drizzle and MySQL/TiDB scaffold | Production persistence path | Provides the project structure for database-backed organizations and records.       |
| Manus WebDev runtime            | Development and preview     | Provides project scaffolding, managed preview, server runtime, and checkpointing.   |

The team can truthfully explain that the interface was designed specifically for this problem, the backend contracts were implemented in TypeScript, the intelligence engine was written as auditable business logic, and the platform provides the development/runtime foundation. No external AI model should be claimed unless it is actually connected and used in the deployed system.

## 1.9 9. Security and privacy

The prototype includes a demo login and logout flow. In production, passwords must never be stored as plain text. The production design should use password hashing, secure session cookies, role-based access control, organization-level data isolation, encryption in transit, file scanning, audit logs, rate limiting, and explicit permissions for imports and exports.

The application should treat uploaded files as potentially sensitive. It should validate file size and type, isolate organization storage, avoid executing uploaded content, record who uploaded a file, and allow administrators to delete or expire source files according to retention policy.

## 1.10 10. File handling policy

The product accepts any file at the upload boundary because organizations do not use one universal format. Automatic analysis is format-aware rather than format-deceptive.

| File class     | Prototype behavior                                                                                                                                   | Production behavior                                                |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| JSON           | The Resource File picker extracts resource rows; the Demand File picker extracts demand rows. Arrays and {resources, demands} objects are supported. | Validate schema and import with an import report.                  |
| CSV / TSV      | Reads headers and maps recognized field names.                                                                                                       | Add column-mapping UI, encoding detection, and validation preview. |
| Plain text     | Attempts table parsing.                                                                                                                              | Add NLP extraction with confidence and human approval.             |
| XLSX           | Accepted as a source file.                                                                                                                           | Add spreadsheet parser and sheet/column selection.                 |
| PDF / DOCX     | Accepted as a source file.                                                                                                                           | Add text/table extraction, OCR when needed, and human review.      |
| Image / scan   | Accepted as a source file.                                                                                                                           | Add OCR and document classification with confidence.               |
| Unknown binary | Accepted without pretending it was analyzed.                                                                                                         | Store metadata, identify format, and route to a safe extractor.    |

## 1.11 11. Why every button exists

The dashboard is intentionally action-oriented. Overview buttons navigate to upload, scenario, analysis, inventory, or demand views. Inventory export downloads the current resource records. Demand intelligence selects a demand and retrieves ranked matches. Daily analysis runs the validation, capacity, matching, and recommendation pipeline. What-if simulation removes a selected resource virtually. Daily Report downloads an evidence summary. Settings provides logout and the product explanation. Help Center provides upload guidance and the team's email addresses.

## 1.12 12. Judge demonstration script

A strong two-to-three minute demonstration should begin at the blank login screen. Sign in with the demo credentials, open **Add your data**, upload a file from the selected domain, and show that the overview changes from zero records to calculated metrics. Open **Demand intelligence** to select one demand and show the best match with the explanation. Open **Daily analysis** to show the pipeline and recommendations. Finally, open **What-if simulator**, remove the highest-impact resource, and explain the affected demands and alternatives. This sequence proves that the system is not a static dashboard with screenshots.

## 1.13 13. Common judge questions and concise answers

### **Q: Is this just an AI dashboard?**

A: No. The core innovation is an intelligence layer that connects existing resource capacity to current demand before acquisition. AI and machine learning are the production roadmap; the prototype uses explainable rules so every result is auditable.

### **Q: How is it different from AWS?**

A: AWS provides cloud infrastructure and managed services. RE:SOURCE is a domain-independent application workflow that analyzes organizational capacity and demand. It can be hosted on AWS, but it solves a different problem at a different layer.

### **Q: How is it different from an ERP?**

A: An ERP records transactions and departmental operations. RE:SOURCE sits across fragmented systems and performs cross-domain reuse, matching, explanation, and what-if decision support.

### **Q: Is the data preloaded?**

A: No. The final prototype starts empty. The user uploads or enters the organization's own records. Every metric and recommendation is computed from those records.

### **Q: What happens if the file format is not CSV?**

A: The uploader accepts any file type. Text-readable structured formats are analyzed automatically in the prototype. Binary or unstructured formats are accepted as source files and are routed to a future extractor or mapping workflow rather than being silently misread.

### **Q: Why not use a black-box model?**

A: A resource allocation recommendation affects money, people, safety, and operations. Showing the evidence behind a score makes the system testable and gives the human decision-maker control. Forecasting and optimization can be added after reliable historical data is available.

### **Q: What is the innovation?**

A: Resource Reuse Before Resource Acquisition, combined with Digital What-If Resource Simulation. The system searches for hidden internal capacity and tests the consequences of losing it.

### **Q: Can it work for colleges, hospitals, agriculture, or government?**

A: Yes. The engine requires generic concepts such as resource, capacity, utilization, demand, location/context, and priority. A college may upload classrooms and lab demands. A hospital may upload beds and department requirements. A logistics company may upload vehicles and routes.

### **Q: How will it scale?**

A: Store tenant-separated data in a database, process large files asynchronously, use a queue for extraction and analysis, add indexes for organization and time, cache summaries, and retain analysis run history. The decision contract remains the same while the ingestion and compute layers scale independently.

### **Q: What is the environmental impact?**

A: Better use of existing equipment, rooms, vehicles, and infrastructure can reduce unnecessary procurement, idle capacity, duplicate expansion, and associated material and energy use. Impact should be measured with organization-specific baseline data rather than claimed without measurement.

## 1.14 14. Limitations to state honestly

The current prototype does not yet provide production-grade persistent multi-tenant storage, full XLSX/PDF/DOCX extraction, password reset, MFA, complete RBAC, IoT connectors, trained forecasting models,

or optimization under all business constraints. These are explicit next-stage engineering tasks. Stating these boundaries improves credibility because the team can distinguish a working proof of concept from a production platform.

## **1.15 15. Team support**

For help with the prototype, contact **[harsimran6080@gmail.com](mailto:harsimran6080@gmail.com)**, **[ayushraj09122009@gmail.com](mailto:ayushraj09122009@gmail.com)**, or **[piyusashish@gmail.com](mailto:piyusashish@gmail.com)**.

## **1.16 References**